

**LAPORAN TUGAS PRAKTIKUM SISTEM OPERASI
DEADLOCK**



Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

Yusa Salasa (2410651018)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

Tugas Praktikum

Tugas 1: Analisis Deadlock

Jalankan program `deadlock_simple.c` dan jelaskan:

1. Mengapa program mengalami deadlock?
2. Kondisi deadlock apa saja yang terpenuhi?
3. Gambar diagram resource allocation graph-nya

Tugas 2: Implementasi Pencegahan

Modifikasi program `deadlock_simple.c` menggunakan teknik:

1. Lock ordering
2. Try-lock dengan retry
3. Bandingkan kedua metode tersebut

Tugas 3: Banker's Algorithm

Buat program baru dengan data :

- 4 proses
- 3 jenis resource
- Available : [5, 4, 3]
- Implementasikan request resource dan cek keamanan sistem

Tugas 4: Kasus Nyata (Real Case)

Buat simulasi deadlock untuk kasus: “Dua orang ingin transfer uang antar rekening secara bersamaan”

- Orang A: transfer dari rekening 1 ke rekening 2
- Orang B: transfer dari rekening 2 ke rekening 1
- Implementasikan deadlock prevention

TUGAS 1 – Analisis Deadlock

1. Mengapa program deadlock_simple.c mengalami deadlock?

Program mengalami deadlock karena kedua thread saling menunggu lock yang sedang dipegang thread lain.

Urutannya:

- Thread 1 mengunci Lock1, lalu mencoba mengambil Lock2 (tapi Lock2 sedang dipegang Thread 2).
- Thread 2 mengunci Lock2, lalu mencoba mengambil Lock1 (tapi Lock1 sedang dipegang Thread 1).

Keduanya menunggu tanpa ada yang melepaskan lock terlebih dahulu.

Akibatnya, tidak ada thread yang bisa melanjutkan, sehingga program berhenti (deadlock).

2. Kondisi Deadlock (Coffman Conditions) yang Terpenuhi

Semua syarat deadlock terpenuhi:

a. Mutual Exclusion

Lock1 dan Lock2 hanya bisa digunakan oleh satu thread pada satu waktu.

b. Hold and Wait

- Thread 1 memegang Lock1 → menunggu Lock2
- Thread 2 memegang Lock2 → menunggu Lock1

c. No Preemption

Lock tidak dapat direbut paksa; thread harus melepasnya sendiri.

d. Circular Wait

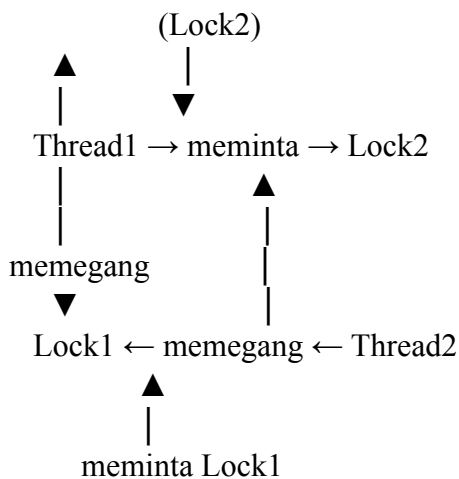
Ada lingkaran tunggu:

Thread 1 → menunggu Lock2 → dipegang Thread 2

Thread 2 → menunggu Lock1 → dipegang Thread 1

Siklus inilah penyebab deadlock.

3. Diagram Resource Allocation Graph (RAG)



Tugas 2: Implementasi Pencegahan Deadlock

1. Menggunakan Lock Ordering

Pada metode ini, semua thread diwajibkan mengambil lock dalam urutan yang sama. Misalnya, selalu mengambil lock1 dulu, baru lock2.

Dengan aturan yang konsisten, situasi saling tunggu dapat dihindari karena tidak ada thread yang mengambil resource dengan urutan terbalik.

Kode Program (Lock Ordering)

```
#include <stdio.h>

#include <pthread.h>

#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void* worker(void* arg) {

    printf("Thread mulai mengambil resource...\n");

    pthread_mutex_lock(&lock1);

    printf("Lock 1 didapat.\n");
```

```
pthread_mutex_lock(&lock2);

printf("Lock 2 didapat.\n");


printf("Thread melakukan pekerjaan...\n");

sleep(1);


pthread_mutex_unlock(&lock2);

pthread_mutex_unlock(&lock1);


printf("Semua resource dilepas.\n");

return NULL;
}

int main() {

    pthread_t t1, t2;

    printf("=== PENCEGAHAN: LOCK ORDERING ===\n");

    pthread_create(&t1, NULL, worker, NULL);

    pthread_create(&t2, NULL, worker, NULL);


    pthread_join(t1, NULL);

    pthread_join(t2, NULL);


    printf("Program selesai tanpa deadlock.\n");

    return 0;
}
```

2. Menggunakan Try-Lock + Retry

Metode ini menggunakan `pthread_mutex_trylock()`.

Jika thread gagal mengambil lock kedua:

- thread akan melepaskan lock pertama,
- menunggu sebentar,
- kemudian mencoba lagi.

Dengan cara ini, kondisi “hold and wait” bisa dihindari.

Kode Program (Try-lock + Retry)

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void* thread_func(void* arg) {
```

```
    while (1) {
```

```
        pthread_mutex_lock(&lock1);
```

```
        printf("Thread: Lock 1 didapat.\n");
```

```
        if (pthread_mutex_trylock(&lock2) == 0) {
```

```
            printf("Thread: Lock 2 berhasil didapat.\n");
```

```
            printf("Thread sedang bekerja...\n");
```

```
            sleep(1);
```

```

        pthread_mutex_unlock(&lock2);

        pthread_mutex_unlock(&lock1);

        break;
    } else {

        printf("Gagal mendapatkan Lock 2, ulangi lagi...\n");

        pthread_mutex_unlock(&lock1);

        usleep(100000);

    }

}

return NULL;
}

int main() {

    pthread_t t1, t2;

    printf("=== PENCEGAHAN: TRY-LOCK + RETRY ===\n");


    pthread_create(&t1, NULL, thread_func, NULL);

    pthread_create(&t2, NULL, thread_func, NULL);


    pthread_join(t1, NULL);

    pthread_join(t2, NULL);


    printf("Program selesai tanpa deadlock.\n");

    return 0;

}

```

3. Perbandingan Kedua Metode

Metode	Cara Kerja	Keunggulan	Kekurangan	Cocok Untuk
Lock Ordering	Semua thread mengikuti urutan pengambilan lock yang sama	Sangat sederhana dan efektif	Harus ada urutan tetap, kurang fleksibel	Resource jumlahnya tetap dan jelas
Try-Lock + Retry	Jika lock kedua gagal → lepas lock pertama → coba lagi	Lebih fleksibel, bisa dipakai saat resource dinamis	Banyak percobaan berulang bisa membebani CPU	Lingkungan dinamis atau kompetitif

TUGAS 3 – Banker's Algorithm

Program ini memakai data:

- 4 proses
- 3 resource
- Available = [5, 4, 3]

Kode Program

```
#include <stdio.h>
```

```
#define P 4
```

```
#define R 3
```

```
int available[R] = {5, 4, 3};
```

```
int max[P][R];
```

```
int alloc[P][R];
```

```
int need[P][R];
```

```
int isSafe() {
```

```
    int work[R];
```

```
    int finish[P] = {0};
```

```
    for (int i = 0; i < R; i++) work[i] = available[i];
```

```
    int done = 0;
```

```
    while (done < P) {
```

```
        int found = 0;
```



```

for (int i = 0; i < P; i++) {
    if (!finish[i]) {
        int ok = 1;
        for (int j = 0; j < R; j++) {
            if (need[i][j] > work[j]) {
                ok = 0;
                break;
            }
        }
        if (ok) {
            for (int j = 0; j < R; j++)
                work[j] += alloc[i][j];
            finish[i] = 1;
            found = 1;
            done++;
        }
    }
}
if (!found) return 0;
}
return 1;
}

```

```

int request(int p, int req[]) {
    for (int j = 0; j < R; j++)
        if (req[j] > need[p][j])
            return -1;

```

```

    for (int j = 0; j < R; j++)
        if (req[j] > available[j])
            return 0;

```

```

    for (int j = 0; j < R; j++) {
        available[j] -= req[j];
        alloc[p][j] += req[j];
        need[p][j] -= req[j];
    }

```

```

    if (isSafe())
        return 1;

```

```

    for (int j = 0; j < R; j++) {
        available[j] += req[j];
        alloc[p][j] -= req[j];
        need[p][j] += req[j];
    }

    return 0;
}

int main() {
    printf("=== Banker's Algorithm ===\n\n");

    printf("Masukkan matriks MAX (4 proses × 3 resource):\n");
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            scanf("%d", &max[i][j]);

    printf("\nMasukkan matriks ALLOCATION (4 proses × 3 resource):\n");
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            scanf("%d", &alloc[i][j]);

    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    while (1) {
        int choice;
        printf("\nMenu:\n1. Request Resource\n2. Cek Safe State\n3. Exit\nPilihan: ");
        scanf("%d", &choice);

        if (choice == 1) {
            int p, req[R];
            printf("Masukkan nomor proses (0–3): ");
            scanf("%d", &p);
            printf("Masukkan request %d resource: ", R);
            for (int j = 0; j < R; j++) scanf("%d", &req[j]);

            int res = request(p, req);

```

```

        if (res == 1)
            printf("Request disetujui. Sistem dalam keadaan aman.\n");
        else if (res == 0)
            printf("Resource tidak cukup. Proses harus menunggu.\n");
        else
            printf("Request melebihi need proses.\n");
    }
    else if (choice == 2) {
        if (isSafe())
            printf("Sistem dalam keadaan AMAN.\n");
        else
            printf("Sistem TIDAK AMAN.\n");
    }
    else if (choice == 3) {
        break;
    }
}
return 0;
}

```

OUTPUT:

```

yos@yos-QEMU-Virtual-Machine:~$ ./Bankers_4proses
=== Banker's Algorithm ===

Masukkan matriks MAX (4 proses x 3 resource):
3 4 1
4 6 3
7 8 4
1 9 5

Masukkan matriks ALLOCATION (4 proses x 3 resource):
5 3 4
4 7 2
9 7 8
4 6 2

Menu:
1. Request Resource
2. Cek Safe State
3. Exit
Pilihan: 1
Masukkan nomor proses (0-3): 4
Masukkan request 3 resource: 3 2 1
Request melebihi need proses.

Menu:
1. Request Resource
2. Cek Safe State
3. Exit
Pilihan: 2
Sistem dalam keadaan AMAN.

Menu:
1. Request Resource
2. Cek Safe State
3. Exit
Pilihan: 1
Masukkan nomor proses (0-3): 1
Masukkan request 3 resource: 1 3 2
Request melebihi need proses.

Menu:
1. Request Resource
2. Cek Safe State
3. Exit
Pilihan: 3
yos@yos-QEMU-Virtual-Machine:~$

```

Tugas 4: Kasus Nyata (Real Case)

Simulasi Kasus

Ada dua orang yang melakukan transfer di waktu yang hampir bersamaan:

- Orang A: transfer dari rekening 1 → rekening 2
- Orang B: transfer dari rekening 2 → rekening 1

Kalau kedua proses ini saling mengunci rekening yang berbeda di awal, maka bisa terjadi deadlock, karena masing-masing menunggu rekening yang masih dikunci proses lain.

Kode dengan Pencegahan Deadlock

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t rekening1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t rekening2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void lock_ordered(pthread_mutex_t *a, pthread_mutex_t *b) {
```

```
    if (a < b) {
```

```
        pthread_mutex_lock(a);
```

```
        pthread_mutex_lock(b);
```

```
    } else {
```

```
        pthread_mutex_lock(b);
```

```
        pthread_mutex_lock(a);
```

```
    }
```

```
}
```

```
void unlock_ordered(pthread_mutex_t *a, pthread_mutex_t *b) {  
  
    pthread_mutex_unlock(a);  
  
    pthread_mutex_unlock(b);  
  
}
```

```
void* transfer_A(void *arg) {  
  
    lock_ordered(&rekening1, &rekening2);  
  
    printf("A: Transfer dari Rekening 1 ke 2\n");  
  
    sleep(1);  
  
    unlock_ordered(&rekening1, &rekening2);  
  
    return NULL;  
  
}
```

```
void* transfer_B(void *arg) {  
  
    lock_ordered(&rekening1, &rekening2);  
  
    printf("B: Transfer dari Rekening 2 ke 1\n");  
  
    sleep(1);  
  
    unlock_ordered(&rekening1, &rekening2);  
  
    return NULL;  
  
}
```

```
int main() {  
  
    pthread_t A, B;
```

```
printf("=== SIMULASI TANPA DEADLOCK ===\n");

pthread_create(&A, NULL, transfer_A, NULL);

pthread_create(&B, NULL, transfer_B, NULL);


pthread_join(A, NULL);

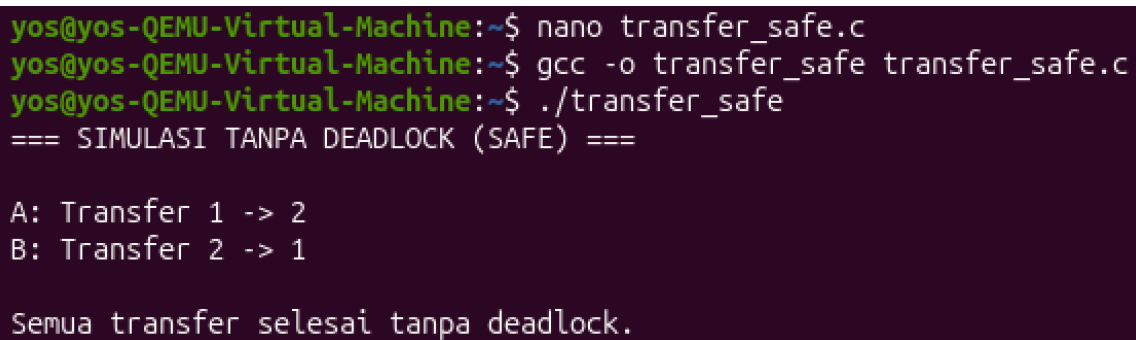
pthread_join(B, NULL);


printf("Semua transaksi selesai.\n");

return 0;

}
```

OUTPUT :



```
yos@yos-QEMU-Virtual-Machine:~$ nano transfer_safe.c
yos@yos-QEMU-Virtual-Machine:~$ gcc -o transfer_safe transfer_safe.c
yos@yos-QEMU-Virtual-Machine:~$ ./transfer_safe
=== SIMULASI TANPA DEADLOCK (SAFE) ===

A: Transfer 1 -> 2
B: Transfer 2 -> 1

Semua transfer selesai tanpa deadlock.
```

Kesimpulan

Dari empat tugas yang dikerjakan, bisa disimpulkan bahwa deadlock terjadi karena proses atau thread saling berebut resource dan saling menunggu tanpa ada yang bisa melanjutkan. Pada program *deadlock_simple.c*, kondisi deadlock jelas muncul karena kedua thread memegang satu lock dan menunggu lock lain, sehingga program berhenti total.

Cara pencegahan yang dicoba — seperti lock ordering dan try-lock — terbukti mampu menghilangkan deadlock dengan memberikan aturan penguncian dan mekanisme mencoba ulang. Pada Banker's Algorithm, sistem bisa mengecek apakah sebuah permintaan resource aman atau berpotensi membuat deadlock. Hasilnya, request hanya disetujui jika sistem tetap berada pada keadaan aman. Untuk kasus transfer uang, deadlock dapat dicegah dengan memberi urutan penguncian yang sama pada kedua proses sehingga tidak terjadi saling tunggu.

Secara keseluruhan, semua percobaan menunjukkan bahwa deadlock bisa dihindari jika resource dikelola dengan aturan yang konsisten.

Saran

1. Sebaiknya gunakan pola penguncian yang sama pada semua thread agar tidak terjadi circular wait.
2. Pada kasus tertentu, *try-lock* atau timeout bisa dipakai supaya proses tidak menunggu terlalu lama.
3. Untuk sistem dengan resource banyak dan kompleks, Banker's Algorithm dapat membantu mencegah deadlock sejak awal.
4. Penggunaan log atau print sederhana sangat membantu untuk melihat alur resource saat debugging.